

Choosing the Right Caching Layer

 systemdesignschool.io/fundamentals/cdn-vs-app-cache



The Edge: CDN vs Application Cache

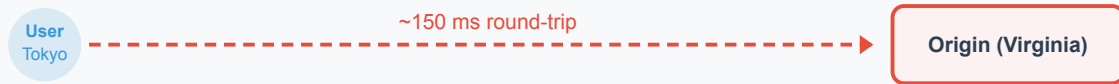
The previous article mapped four caching layers. In practice, the critical architectural decision is the boundary between CDN caching and application caching. Both reduce origin load. On a cache hit, both serve without hitting the origin database. But they solve different problems, and choosing wrong means either serving stale personalized data or wasting infrastructure on requests a CDN could have absorbed.

The Geographic Problem

A user in Tokyo requests a product image. Without a CDN, the request travels to the origin server in Virginia—a round trip of roughly 150ms dictated by the speed of light through fiber. With a CDN, the request hits a Tokyo edge server and returns at local-network latency.

CDN Edge vs Direct Origin

Without CDN: Every request travels to origin



With CDN: Edge serves from nearby cache



Latency improvement: 150 ms → 10 ms (15x faster)

Origin only handles cache misses, not every request



This latency gap is physics, not engineering. No amount of server optimization eliminates the speed-of-light delay across the Pacific. A CDN solves it by placing cached copies at the network edge, geographically close to users.

What CDNs Handle Well

CDNs cache at the HTTP level. They store complete HTTP responses (headers + body) keyed primarily by URL (plus host and selected headers via **Vary**). This works naturally for content that is:

Public and identical for all users. A product image, a JavaScript bundle, a marketing page. Every user receives the same bytes. Each CDN edge stores a local copy and serves it to nearby users without hitting the origin.

Rarely changing. A CSS file changes on deployment, not between requests. The CDN can cache it for days or weeks. When it changes, you deploy with a new filename (content-hashed URLs) so the old cache is never stale—it's simply never requested again.

Tolerance for staleness. A blog post cached for 5 minutes at the edge means a typo fix takes 5 minutes to propagate globally. For most static content, this is acceptable.

CDNs are the right choice for images, videos, fonts, compiled assets, documentation pages, and any content where the response is the same regardless of who requested it.

Where CDNs Fall Short

CDNs struggle when responses vary by user, session, or real-time state.

Personalized responses. A user's feed, cart, or notification count differs per user. Caching at the CDN means either serving everyone the same generic response (losing personalization) or maintaining billions of per-user cache entries at every edge location (impractical).

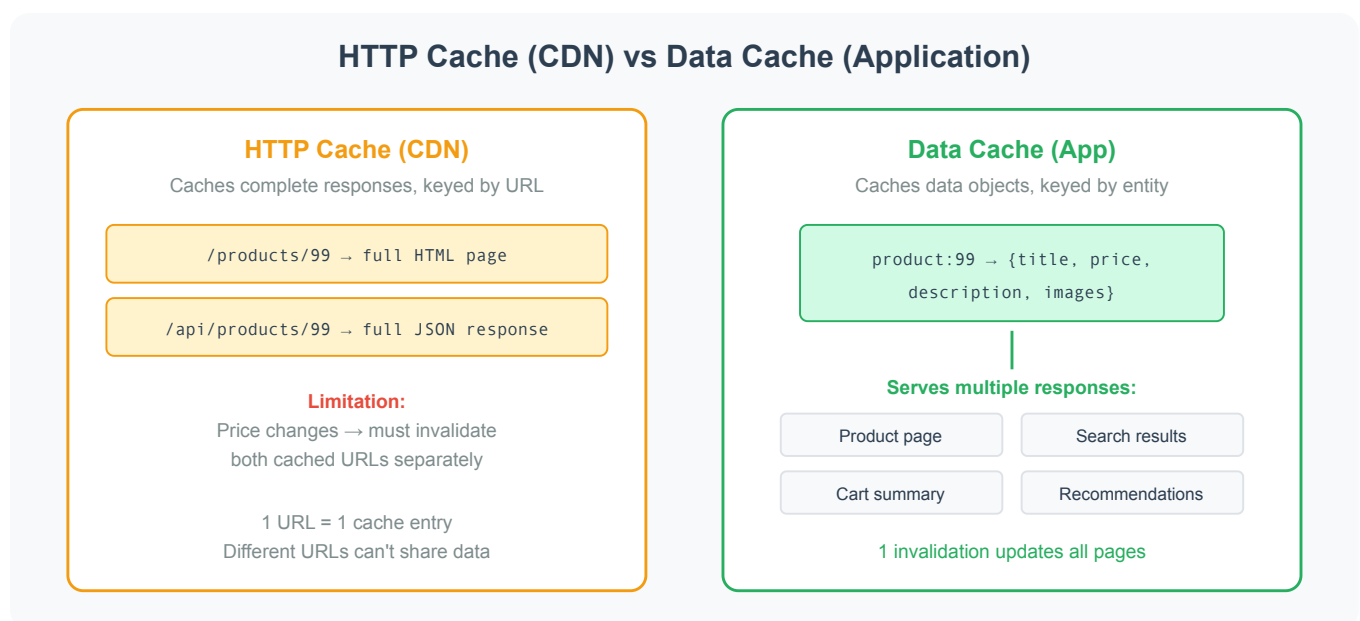
High cardinality keys. A search API with millions of possible query combinations has a poor cache hit rate at any single edge location. The edge server in Tokyo might see each unique query only once, making caching pointless.

Consistency-sensitive data. Inventory counts, auction bids, and account balances need real-time accuracy. Even a 5-second CDN TTL means a user might buy an item that's already sold out.

For these cases, the application cache is the right layer. It sits behind the CDN, closer to the application servers, and caches at the data level rather than the HTTP level.

Application Cache: Data-Level Caching

An application cache (typically an in-memory store like Redis or Memcached) stores individual data objects—a user record, a product's metadata, a computed feed—rather than complete HTTP responses.

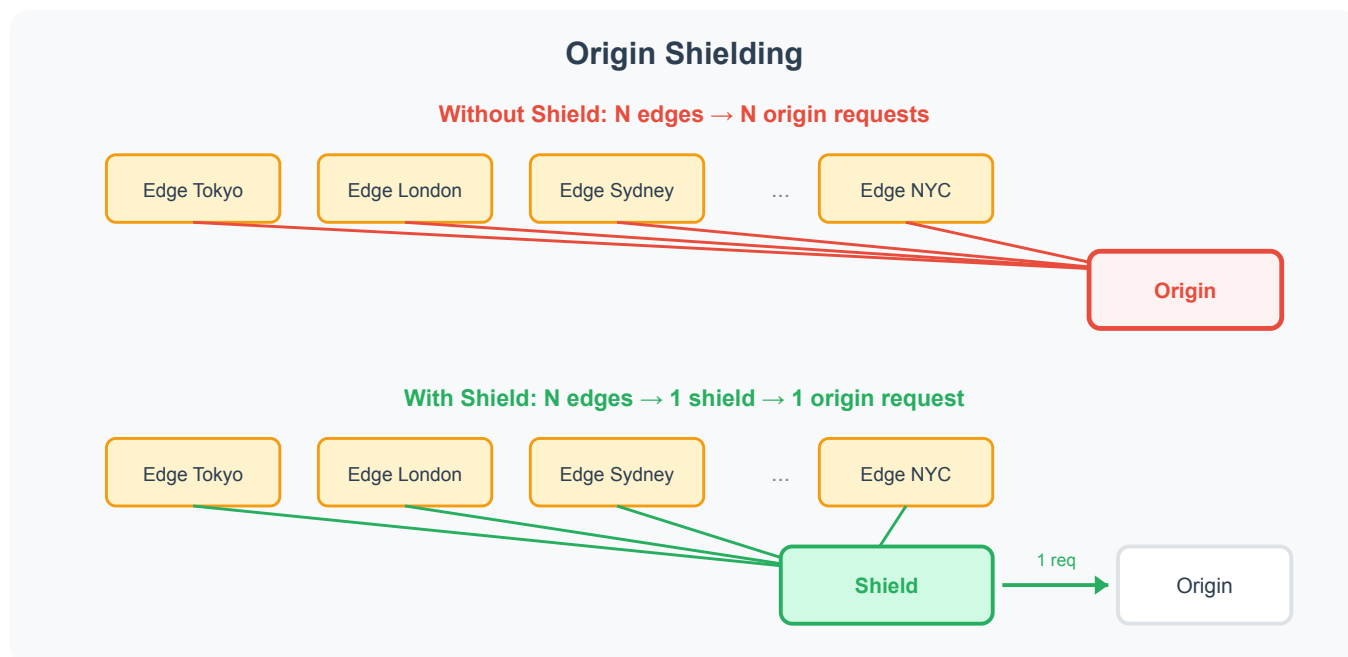


This distinction matters. An HTTP-cached page couples the cache key to the URL. A data-cached object can be assembled into any response. The same cached user profile serves the profile page, the settings page, and the activity feed. One invalidation updates all three.

Application caching handles what CDNs cannot: per-user data, complex aggregations, and computed results that don't map cleanly to a single URL.

Origin Shielding

When a CDN cache misses, it fetches from the origin. If you have 200 edge servers worldwide and a popular page expires simultaneously at all of them, 200 requests hit your origin at once. With thousands of URLs, this becomes a continuous barrage.



Origin shielding adds a caching layer between the CDN edge and your origin. Instead of 200 edge servers each fetching from your origin, they route through a designated shield POP (or small set of shield POPs). The shield either has the content cached or fetches it from the origin once per object.

The result: your origin sees far fewer requests—often close to one per object per shield—instead of one per edge. The shield absorbs the fan-in from global edge servers. This is particularly valuable during cache expiration events or traffic spikes when many edges simultaneously discover their caches are stale.

Most CDN providers offer origin shielding as a configuration option. In system design interviews, mention it when discussing CDN architecture for high-traffic systems—it demonstrates awareness of the thundering herd problem at the edge layer.

Decision Framework

Use this framework to decide which layer handles each type of data:

Data Type	CDN	App Cache	Why
Static assets (JS, CSS, images)	Yes	No	Same for all users, changes rarely

Data Type	CDN	App Cache	Why
Public pages (marketing, docs)	Yes	No	Identical responses, tolerate staleness
API responses (personalized)	No	Yes	Vary by user, need data-level control
Hot database objects	No	Yes	Fine-grained invalidation needed
Semi-static API (product catalog)	Yes	Yes	CDN for pages, app cache for partial updates

"Yes" in the CDN column assumes the response is public and identical for all users. Personalized or auth-gated responses typically bypass the CDN (or require careful cache-key variation) even if the data type seems static.

For most systems, the split is straightforward: CDNs for static assets and public pages, application cache for everything else. The interesting cases are hybrid—a product page where the static layout shell is CDN-cached but personalized fields (price, inventory, recommendations) are fetched separately from the application cache. This "fragment" approach lets each piece expire independently: the shell lives at the CDN for hours while dynamic data refreshes every few seconds from the app cache.

What's Next

The previous articles covered what caching is and where caches live. The remaining articles dive into implementation: how data enters the cache (read patterns), how it stays fresh (write patterns and invalidation), and what happens when things go wrong (failure modes). From here, cache key design shows how to structure the keys that identify cached data.